PSE EM AÇÃO - SISTEMA DE PLANEJAMENTO E ACOMPANHAMENTO DE AÇÕES DE SAÚDE NA ESCOLA

Erick Lohan Naresi Mariano – RA: 26003481-2; Pedro Augusto dos Santos Rodrigues – RA: 26003135-2; Vitor Hugo Oliveira Tanaka – RA: 26004113-2

ESOFT2SNB – 2ª ENTREGA

---

1. APRESENTAÇÃO DO PROJETO E VÍDEO DE DEMONSTRAÇÃO

O projeto PSE em Ação foi concebido para atender às demandas de planejamento e acompanhamento intersetorial do Programa Saúde na Escola (PSE), estabelecido pelo Decreto nº 6.286/2007. Esta segunda entrega apresenta a implementação computacional completa desenvolvida em Linguagem C, acompanhada dos artefatos de Engenharia de Software atualizados, testes funcionais, manual do usuário e vídeo demonstrativo.

Repositórios GitHub:

https://github.com/vitorhugotanaka/AEP.2SEMESTRE.PRIMEIRA_ENTREGA

https://github.com/vitorhugotanaka/AEP.2SEMESTRE.SEGUNDA_ENTREGA

1.1 ACESSO À DEMONSTRAÇÃO EM VÍDEO (APRESENTAÇÃO ORAL - MÁX. 7 MINUTOS)

O vídeo completo de demonstração da aplicação, explicação do código-fonte C, navegação pelos casos de uso e integração com o planejamento incremental da disciplina encontra-se disponível no link com acesso liberado para avaliação:

URL do Vídeo: https://youtu.be/-1-_XDB7ir0?is=Ig6TPJZ0_hrq-EqM

---

2. ARTEFATOS DE ENGENHARIA DE SOFTWARE

2.1 MATRIZ DE REQUISITOS FUNCIONAIS E NÃO FUNCIONAIS (ATUALIZADA)

Durante a fase de implementação em C, os requisitos originais foram refinados para assegurar total robustez no tratamento de exceções em terminal.

Código	Tipo	Descrição do Requisito	Status na Entrega
RF01	Funcional	Cadastrar ação do PSE informando código único, escola, tema, data prevista, público-alvo, responsável, quantidade prevista de participantes e situação inicial 'Planejada'.	Implementado
RF02	Funcional	Listar todas as ações cadastradas exibindo de forma organizada todos os atributos armazenados em memória.	Implementado
RF03	Funcional	Pesquisar ações por código exato, substring do nome da escola ou substring do tema cadastrado.	Implementado
RF04	Funcional	Atualizar a situação da ação (1-Planejada, 2-Realizada, 3-Cancelada). Se 'Realizada', registrar a quantidade efetiva de participantes.	Implementado
RF05	Funcional	Gerar resumo estatístico comparando total de ações por status, total de participantes previstos vs. efetivos e percentual de adesão.	Implementado
RF06	Funcional	Validar entradas do usuário impedindo códigos nulos/negativos, códigos duplicados, estouro de vetor e quantidades negativas.	Implementado
RNF01	Não Funcional	Execução nativa em ambiente de terminal sem dependências de SGBD ou bibliotecas gráficas externas.	Atendido
RNF02	Não Funcional	Interface amigável via console com mensagens claras de erro, alertas e confirmações estruturadas.	Atendido
RNF03	Não Funcional	Garantia de privacidade e conformidade ética: restrição estrita a dados fictícios coletivos (sem prontuários ou diagnósticos).	Atendido

2.2 DIAGRAMA E ESPECIFICAÇÃO DOS CASOS DE USO

O sistema interage com um ator principal: o Profissional da Equipe Intersetorial IntegraPSE.

Caso de Uso	Ator Principal	Resumo da Interação
UC01 - Cadastrar Ação	Profissional IntegraPSE	Insere os dados estruturados de uma nova ação de saúde no sistema. O sistema valida código único e valores não negativos.
UC02 - Listar Ações	Profissional IntegraPSE	Exibe a listagem completa de todas as ações armazenadas no vetor de memória com formatação clara.
UC03 - Pesquisar Ações	Profissional IntegraPSE	Permite ao usuário buscar registros filtrando por código, nome da escola ou tema da atividade.
UC04 - Atualizar Situação	Profissional IntegraPSE	Altera o estado operacional da ação (Planejada/Realizada/Cancelada). Ao marcar 'Realizada', solicita a presença efetiva.
UC05 - Gerar Resumo Geral	Profissional IntegraPSE	Calcula estatísticas consolidadas da rede escolar e exibe o percentual global de alcance.

2.2.1 DETALHAMENTO COMPLETO DO UC04: ATUALIZAR SITUAÇÃO DA AÇÃO

Ator: Profissional

Pré-condição: existir ao menos uma ação cadastrada no vetor em memória.

Fluxo Principal:

1. O usuário seleciona a Opção 4 no menu principal.

2. O sistema solicita o código da ação desejada.

3. O usuário informa o código (ex: 101).

4. O sistema busca a ação no vetor, encontra e exibe seus dados atuais.

5. O sistema solicita a nova situação (1 - Planejada, 2 - Realizada, 3 - Cancelada).

6. O usuário escolhe "2 - Realizada".

7. O sistema solicita a quantidade efetiva de participantes.

8. O usuário digita um valor válido (ex: 45).

9. O sistema atualiza a struct correspondente, registra a situação e exibe a confirmação de sucesso.

Fluxos Alternativos / Exceções:

FA01 (Código Não Encontrado): O sistema exibe [ERRO] Ação com código X não encontrada e retorna ao menu.

FA02 (Quantidade Efetiva Negativa): O sistema exibe [ERRO] Quantidade inválida e repete a leitura até receber um valor ≥ 0.

Pós-condição: a ação possui seu status e dados de participação efetiva devidamente atualizados na memória.

---

3. PLANO DE TESTES INTEGRADOS E RESULTADOS

Abaixo apresenta-se a suíte de testes executada no ambiente de validação:

ID Teste	Caso de Teste / Entradas	Resultado Esperado	Resultado Obtido	Status
CT-01	Cadastrar ação com código duplicado (Código 101).	Mensagem de erro [ERRO] Já existe uma ação... e bloqueio do cadastro.	Erro exibido corretamente sem alterar o vetor.	APROVADO
CT-02	Cadastrar ação com quantidade prevista de -15 participantes.	Alerta de quantidade inválida e re-solicitação da entrada.	Laço do-while reteve o fluxo até digitação de valor positivo.	APROVADO
CT-03	Pesquisar por substring de escola ("Colegio").	Retornar todas as ações cujas escolas contêm a palavra.	Busca via strstr exibiu com precisão as 2 ações correspondentes.	APROVADO
CT-04	Atualizar situação para Realizada e informar 50 participantes.	Alterar status da struct e gravar qtd_efetiva = 50.	Dados gravados e refletidos na listagem e no resumo geral.	APROVADO
CT-05	Digitar letra 'A' no menu principal numérico.	Limpeza de buffer via limparBuffer() e mensagem de opção inexistente.	Sistema manteve estabilidade sem entrar em loop infinito.	APROVADO

---

4. MANUAL DO USUÁRIO E DEMONSTRAÇÃO OPERACIONAL

4.1 INSTALAÇÃO, COMPILAÇÃO E EXECUÇÃO

O sistema foi desenvolvido em Linguagem C ANSI/C99 standard, garantindo total portabilidade para ambientes GCC, Clang ou MSVC em Linux, Windows e macOS.

Compilação via GCC (Terminal): gcc main.c -o pse_em_acao

Execução (Linux/macOS): ./pse_em_acao

Execução (Windows): pse_em_acao.exe

4.2 CAPTURAS DE TELA DE EXECUÇÃO (SIMULAÇÃO DE TERMINAL)

Menu Principal do Sistema (Opção 1 a 6):

```
=== SISTEMA INTEGRAPSE - MENU PRINCIPAL ===
1. Cadastrar acao
2. Listar acoes
3. Pesquisar acao
4. Atualizar situacao
5. Gerar resumo geral
6. Encerrar
Digite a opcao desejada: 1
```

Execução do Cadastro de Ação (com validação de dados):

```
--- CADASTRO DE NOVA ACAO ---
Digite o codigo da acao (numero inteiro positivo): 101
Digite o nome da escola: Esc. Municipal Monteiro Lobato
Digite o tema (ex: Alimentacao Saudavel / Atividade Fisica): Alimentacao Saudavel
Digite a data prevista (DD/MM/AAAA): 15/09/2026
Digite o publico-alvo: Alunos do 5º Ano
Digite o responsavel: Nutricionista Ana Paula
Digite a quantidade prevista de participantes: -10
[ERRO] Quantidade invalida! Digite um valor maior ou igual a zero.
Digite a quantidade prevista de participantes: 50
[SUCESSO] Acao codigo 101 cadastrada com sucesso!
```

Listagem de Ações Cadastradas (Opção 2):

```
================ LISTAGEM DE ACOES (2) ================
------------------------------------------------
Codigo: 101
Escola: Esc. Municipal Monteiro Lobato
Tema: Alimentacao Saudavel
Data Prevista: 15/09/2026
Publico-Alvo: Alunos do 5º Ano
Responsavel: Nutricionista Ana Paula
Situacao: Realizada
Qtd Prevista: 50
Qtd Efetiva: 48
------------------------------------------------
Codigo: 102
Escola: Col. Estadual Cora Coralina
Tema: Atividade Fisica e Saude
```

```
Data Prevista: 20/09/2026
Publico-Alvo: Turmas do 8º Ano
Responsavel: Prof. Roberto Silva
Situacao: Planejada
Qtd Prevista: 100
--------------------------------------------------
```

Geração do Resumo Geral de Indicadores (Opção 5):

```
================ RESUMO GERAL DAS AÇÕES PSE ================
Total de acoes cadastradas: 2
 - Acoes Planejadas: 1
 - Acoes Realizadas: 1
 - Acoes Canceladas: 0
------------------------------------------------------------
Total de participantes previstos: 150
Total de participantes efetivos: 48
Percentual de participacao: 32.00%
============================================================
```

---

5. CÓDIGO-FONTE COMPLETO DA APLICAÇÃO (MAIN.C)

Segue abaixo o código-fonte integral em C implementado pelo grupo, modularizado com funções dedicadas e estruturas de controle adequadas.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <locale.h>

#define MAX_ACOES 100

typedef struct {
    int codigo;
    char escola[100];
    char tema[100];
    char data_prevista[20];
    char publico_alvo[100];
    char responsavel[100];
    int qtd_prevista;
    int qtd_efetiva;
    int situacao; // 1: Planejada, 2: Realizada, 3: Cancelada
} Acao;

Acao acoes[MAX_ACOES];
int total_acoes = 0;
```

```
void limparBuffer() {
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}

int buscarPorCodigo(int codigo) {
    for (int i = 0; i < total_acoes; i++) {
        if (acoes[i].codigo == codigo) return i;
    }
    return -1;
}

void cadastrarAcao() {
    if (total_acoes >= MAX_ACOES) {
        printf("\n[ERRO] Limite maximo de acoes atingido (%d).\n", MAX_ACOES);
        return;
    }
    Acao nova;
    printf("\n--- CADASTRO DE NOVA ACAO ---\n");
    printf("Digite o codigo da acao (numero inteiro positivo): ");
    if (scanf("%d", &nova.codigo) != 1 || nova.codigo <= 0) {
        printf("[ERRO] Codigo invalido! Must be positive.\n");
        limparBuffer(); return;
    }
    limparBuffer();
    if (buscarPorCodigo(nova.codigo) != -1) {
        printf("[ERRO] Ja existe uma acao cadastrada com o codigo %d.\n", nova.codigo);
        return;
    }
    printf("Digite o nome da escola: ");
    fgets(nova.escola, sizeof(nova.escola), stdin);
    nova.escola[strcspn(nova.escola, "\n")] = 0;

    printf("Digite o tema (ex: Alimentacao Saudavel / Atividade Fisica): ");
    fgets(nova.tema, sizeof(nova.tema), stdin);
    nova.tema[strcspn(nova.tema, "\n")] = 0;

    printf("Digite a data prevista (DD/MM/AAAA): ");
    fgets(nova.data_prevista, sizeof(nova.data_prevista), stdin);
    nova.data_prevista[strcspn(nova.data_prevista, "\n")] = 0;

    printf("Digite o publico-alvo: ");
    fgets(nova.publico_alvo, sizeof(nova.publico_alvo), stdin);
    nova.publico_alvo[strcspn(nova.publico_alvo, "\n")] = 0;

    printf("Digite o responsavel: ");
```

```
    fgets(nova.responsavel, sizeof(nova.responsavel), stdin);
    nova.responsavel[strcspn(nova.responsavel, "\n")] = 0;

    do {
        printf("Digite a quantidade prevista de participantes: ");
        if (scanf("%d", &nova.qtd_prevista) != 1 || nova.qtd_prevista < 0) {
            printf("[ERRO] Quantidade invalida! Digite um valor >= 0.\n");
            limparBuffer();
            nova.qtd_prevista = -1;
        }
    } while (nova.qtd_prevista < 0);
    limparBuffer();

    nova.situacao = 1; // Planejada
    nova.qtd_efetiva = 0;
    acoes[total_acoes] = nova;
    total_acoes++;
    printf("\n[SUCESSO] Acao codigo %d cadastrada com sucesso!\n", nova.codigo);
}

void exibirAcao(Acao a) {
    char str_situacao[20];
    if (a.situacao == 1) strcpy(str_situacao, "Planejada");
    else if (a.situacao == 2) strcpy(str_situacao, "Realizada");
    else if (a.situacao == 3) strcpy(str_situacao, "Cancelada");
    else strcpy(str_situacao, "Desconhecida");
    printf("------------------------------------------------\n");
    printf("Codigo: %d\nEscola: %s\nTema: %s\nData Prevista: %s\nPublico-Alvo:
    %s\nResponsavel: %s\nSituacao: %s\nQtd Prevista: %d\n",
        a.codigo, a.escola, a.tema, a.data_prevista, a.publico_alvo, a.responsavel, str_situacao,
        a.qtd_prevista);
    if (a.situacao == 2) printf("Qtd Efetiva: %d\n", a.qtd_efetiva);
}

void listarAcoes() {
    if (total_acoes == 0) {
        printf("\n[INFO] Nenhuma acao cadastrada ate o momento.\n");
        return;
    }
    printf("\n================ LISTAGEM DE ACOES (%d) ================\n",
    total_acoes);
    for (int i = 0; i < total_acoes; i++) exibirAcao(acoes[i]);
    printf("------------------------------------------------\n");
}

void pesquisarAcoes() {
    if (total_acoes == 0) {
```

```
        printf("\n[INFO] Nenhuma acao cadastrada para pesquisa.\n");
        return;
    }
    int opcao;
    printf("\n--- PESQUISAR AÇÕES ---\n1. Por Codigo\n2. Por Escola\n3. Por Tema\nOpcao:
    ");
    if (scanf("%d", &opcao) != 1) { limparBuffer(); return; }
    limparBuffer();
    int encontrados = 0;
    if (opcao == 1) {
        int cod;
        printf("Digite o codigo: ");
        if (scanf("%d", &cod) == 1) {
            limparBuffer();
            int idx = buscarPorCodigo(cod);
            if (idx != -1) { exibirAcao(acoes[idx]); encontrados++; }
        }
    } else if (opcao == 2 || opcao == 3) {
        char termo[100];
        printf("Digite o termo de busca: ");
        fgets(termo, sizeof(termo), stdin);
        termo[strcspn(termo, "\n")] = 0;
        for (int i = 0; i < total_acoes; i++) {
            char *alvo = (opcao == 2) ? acoes[i].escola : acoes[i].tema;
            if (strstr(alvo, termo) != NULL) {
                exibirAcao(acoes[i]);
                encontrados++;
            }
        }
    }
    if (encontrados == 0) printf("[INFO] Nenhum registro encontrado.\n");
}

void atualizarSituacao() {
    if (total_acoes == 0) { printf("\n[INFO] Nenhuma acao para atualizar.\n"); return; }
    int cod;
    printf("\nDigite o codigo da acao a atualizar: ");
    if (scanf("%d", &cod) != 1) { limparBuffer(); return; }
    limparBuffer();
    int idx = buscarPorCodigo(cod);
    if (idx == -1) { printf("[ERRO] Acao %d nao encontrada.\n", cod); return; }
    printf("\nEscolha a nova situacao (1-Planejada, 2-Realizada, 3-Cancelada): ");
    int nova_sit;
    if (scanf("%d", &nova_sit) != 1 || nova_sit < 1 || nova_sit > 3) { limparBuffer(); return; }
    limparBuffer();
    acoes[idx].situacao = nova_sit;
    if (nova_sit == 2) {
        do {
```

```
            printf("Digite a quantidade efetiva de participantes: ");
            if (scanf("%d", &acoes[idx].qtd_efetiva) != 1 || acoes[idx].qtd_efetiva < 0) {
                limparBuffer(); acoes[idx].qtd_efetiva = -1;
            }
        } while (acoes[idx].qtd_efetiva < 0);
        limparBuffer();
    }
    printf("\n[SUCESSO] Situacao da acao %d atualizada!\n", cod);
}

void gerarResumoGeral() {
    if (total_acoes == 0) { printf("\n[INFO] Nenhuma acao cadastrada.\n"); return; }
    int plan = 0, real = 0, canc = 0, tot_prev = 0, tot_efet = 0;
    for (int i = 0; i < total_acoes; i++) {
        tot_prev += acoes[i].qtd_prevista;
        if (acoes[i].situacao == 1) plan++;
        else if (acoes[i].situacao == 2) { real++; tot_efet += acoes[i].qtd_efetiva; }
        else if (acoes[i].situacao == 3) canc++;
    }
    float pct = tot_prev > 0 ? ((float)tot_efet / tot_prev) * 100.0f : 0.0f;
    printf("================ RESUMO GERAL DAS AÇÕES PSE ================\n");
    printf("Total de Acoes: %d (Planejadas: %d | Realizadas: %d | Canceladas: %d)\n",
    total_acoes, plan, real, canc);
    printf("Participantes Previstos: %d | Efetivos: %d | Alcance: %.2f%%\n", tot_prev, tot_efet,
    pct);
    printf("============================================================\n");
}

int main() {
    setlocale(LC_ALL, "Portuguese");
    int opcao;
    do {
        printf("\n=== SISTEMA INTEGRAPSE - MENU PRINCIPAL ===\n");
        printf("1. Cadastrar acao\n2. Listar acoes\n3. Pesquisar acao\n4. Atualizar situacao\n5.
        Gerar resumo geral\n6. Encerrar\nOpcao: ");
        if (scanf("%d", &opcao) != 1) { limparBuffer(); continue; }
        limparBuffer();
        switch (opcao) {
            case 1: cadastrarAcao(); break;
            case 2: listarAcoes(); break;
            case 3: pesquisarAcoes(); break;
            case 4: atualizarSituacao(); break;
            case 5: gerarResumoGeral(); break;
            case 6: printf("\nEncerrando sistema PSE...\n"); break;
            default: printf("[ERRO] Opcao invalida.\n"); break;
        }
    } while (opcao != 6);
    return 0;
```

```
}
```

---

6. CONSIDERAÇÕES FINAIS E APRENDIZADOS

O desenvolvimento da segunda entrega consolidou a integração entre os conceitos de Engenharia de Software e a implementação prática em Linguagem C. A estruturação modular permitiu manter o código legível e facilidade de manutenção.

Principais Aprendizados: a manipulação segura de entrada de dados em terminal exigiu atenção especial para limpeza do buffer de teclado (limparBuffer()) e prevenção de estouros de vetores. A aplicação prática dos requisitos funcionais em structs demonstrou como a teoria de Engenharia de Software orienta diretamente a escrita de algoritmos limpos.

Melhorias Futuras: para versões futuras do sistema IntegraPSE, recomenda-se a implementação de persistência de dados em arquivo texto/binário (fopen, fwrite), permitindo a conservação dos dados cadastrados após o encerramento do programa.